

Guida Lab — FI2 02x: Tipi base & dintorni (Esercitazione autonoma)

Corso: Fondamenti di Informatica T-2 (12 CFU · S1) **Materiale:** materiali/slide/02x-x1-Esercitazione Tipi base.pdf — E. Denti, *Tipi base & dintorni*, a.a. 2023/24, 42 slide · materiali/slide/02z-Addendum-Main in Java21.pdf, 7 slide **Modalità:** **guidata** (esercitazione x, mappa in percorso.md) **Prerequisiti di teoria:** 02 Linguaggio e piattaforma — (in corso) **in corso, non chiuso**

⚠ **Prerequisito non chiuso.** Il modulo 02 è aperto dal 2026-09-16: la lezione è studiata e verificata a voce, ma restano tre punti deboli, e due di essi sono *esattamente* ciò che questa esercitazione allena. Non è un ostacolo: 02x è la pratica che chiude 02. Non è materiale nuovo — le slide 2-5 sono dichiaratamente un **riassunto** di 02.

Non esiste startkit per 02x: è un'esercitazione autonoma su slide, niente zip né test JUnit. Il contratto, per l'esercizio finale, è la **lista di collaudo del docente** (§5.2) — quella lista ha lo stesso ruolo che i test dello startkit hanno nei LAB.

Setup

Nessun Eclipse, nessun progetto da importare: 02x si fa da **terminale** e in **jshell**. La macchina è già pronta — verificalo:

```
java -version
jshell --version
```

Output atteso: openjdk version "21.0.12.1" e jshell 21.0.12.1. Se rispondono, hai il **JDK** (non il solo JRE): javac e jshell sono strumenti di sviluppo, e questa è la distinzione 01/02 che al ripasso di stamattina non hai nominato.

Crea una cartella di lavoro **fuori dal repo**, così i .class non finiscono in git:

```
mkdir -p ~/lavoro/fi2/02x && cd ~/lavoro/fi2/02x
```

Anatomia dei due strumenti che userai.

	jshell	javac + java
<i>cosa fa</i>	ambiente interattivo, accetta singole istruzioni senza classe né main	compila un .java in .class, poi la JVM esegue la classe
<i>perché qui</i>	gli esperimenti sui tipi (§2, §3, §4): vedi il risultato e il tipo dedotto subito	l'esercizio finale (§5) prende argomenti dalla riga di comando , e quelli in jshell non esistono
<i>come si esce</i>	/exit	—

jshell è dichiarato dalla fonte: «per prove rapide in Java si può anche usare jshell — ambiente interattivo, accetta singole istruzioni» [fonte: 02x, sl. 13].

Perché in questa guida spesso l'output atteso non c'è. Nei §2, §3, §4 e §6 la guida ti dà i comandi e *non* il risultato: lo scrivi tu prima di eseguire, e jshell fa da correttore. È deliberato, non una dimenticanza. La ragione è il pattern trasversale n. 5 — davanti a una relazione con un verso, leggere la risposta e trovarla ragionevole non distingue chi ha il verso giusto da chi ce l'ha invertito; scriverla prima sì. Dove l'output serve come conferma che l'ambiente funziona (§1, Setup) invece c'è.

Nota su jshell e gli errori di compilazione. Molti esercizi qui sotto servono a farti vedere un **rifiuto del compilatore**. jshell lo mostra come error: con la freccia sotto il punto colpevole, e *non* interrompe la sessione: puoi continuare a scrivere. È il comportamento che vuoi — un errore atteso è un risultato, non un guasto.

Il dominio

L'esercitazione non ha un dominio applicativo: ha un **oggetto di studio**, cioè i tipi base di Java visti da chi arriva dal C. La domanda di fondo che il docente pone, slide dopo slide, è una sola:

Il compilatore accetta questa riga, sì o no? E se la accetta, il risultato è quello che ti aspetti?

Sono due domande diverse, e vanno tenute separate — è la prima △ di questa guida.

△ **Distinzione da non collassare** (profilo/errori.md, trasversale n. 1). «Il compilatore lo rifiuta» e «il risultato è sbagliato» sono due esiti **indipendenti**: - rifiutato dal compilatore → il .class non nasce, non esegui niente (§2); - accettato e risultato sbagliato → compila, gira, e ti dà un numero che non ti aspetti (§3, l'overflow).

Il caso limite che li separa è la coppia della slide 23-24: `short d = c+1;` viene **rifiutato**, mentre `b++` sullo stesso valore al limite viene **accettato** e va in overflow. Stesso fenomeno numerico, due esiti opposti del compilatore. È la stessa struttura di failure/error/non-compila che hai solido dal modulo 01: tre esiti, tre posti dove guardare.

I termini del dominio, dalla fonte

- **tipo primitivo**: `int`, `short`, `long`, `byte`, `char`, `float`, `double`, `boolean`. «Non sono tipi di oggetti → non si può chiedere a un valore primitivo di “fare qualcosa”» [fonte: 02x, sl. 32].
- **classe-libreria**: `Integer`, `Long`, `Character`, `Float`, `Double`... «classi “libreria” di nome simile a tutti i tipi primitivi», che offrono le funzioni statiche che al primitivo non puoi chiedere. «NON VANNO CONFUSE COI TIPI-OGGETTO DI Scala e Kotlin!» [fonte: 02x, sl. 32].

- **costante** (letterale) e il suo **suffisso**: L per long, F per float, M per il decimal di C# [fonte: 02x, sl. 14, 20].
- **perdita di informazione**: il criterio con cui Java decide se un assegnamento è lecito [fonte: 02x, sl. 21].

Le domande di analisi, da tenere aperte

La fonte le pone senza rispondere. Rispondile **tu**, a voce, prima di aprire il terminale:

1. Perché il main di Java è static, e perché è public? public perchè deve essere accessibile dal resto delle classi, static booooh
2. Perché in C gli argomenti da riga di comando includono il nome del programma e in Java no? perchè in java il focus è su chi svolge l'azione prima che quale azione si svolge
3. Perché per stampare in C serve `#include <stdio.h>` e in Java non serve includere nulla? boh

La terza ha una risposta che viene dritta dal modulo 01, e la fonte la dà in due parole: «in Java, non si include nulla (**link dinamico**): si invoca il servizio `println` del componente `System.out`» [fonte: 02x, sl. 5]. Se l'espressione «link dinamico» qui non ti dice subito *cosa non c'è dentro il tuo .class*, torna a appunti_01... §8.1 prima di proseguire: è il verso che stamattina è caduto.

Ordine di lavoro

Dalla progressione della fonte, dal più semplice al più complesso:

§	Esercizio	Cosa allena	Slide
1	Il main e gli argomenti	riprendere 02 in pratica	2-12
2	Assegnamenti fra tipi numerici	il verso della perdita di informazione	20-21
3	Overflow e aritmetica IEEE-754	compilatore accetta $\neq$ risultato giusto	22-31
4	Caratteri, Unicode e UTF	char contro byte	33-36
5	Primitivi contro classi-libreria	a chi si chiede il servizio	32, 37-44
6	Equazioni di 2° grado	l'esercizio completo, con collaudo	45-50

Comincia dal §2, non dal §1. Il §1 è ripasso di 02 e lo puoi fare in dieci minuti, ma il §2 è il punto che ha fatto cadere la verifica del 16/09 e il ripasso di stamattina, e conviene affrontarlo con la testa fresca. Il §1 resta come verifica che la toolchain gira.

§1 — Il main e gli argomenti da riga di comando

Obiettivo: un programma che stampa i propri argomenti, compilato e lanciato a mano. Al termine sai che la toolchain funziona e hai rivisto in pratica `javac/java`.

Concetto minimo. Il `main` di Java sta dentro una classe, è `static` «perché deve esistere dall'inizio alla fine del programma» ed è `public` «perché deve essere visibile dall'esterno» [fonte: 02x, sl. 2]. Riceve «un unico argomento: un array di `String`» [fonte: 02x, sl. 4], e a differenza del C **non** riceve il nome del programma [fonte: 02x, sl. 3].

La fonte mostra questo scheletro, ed è il costrutto nuovo di cui hai diritto al frammento:

```
public class Esempio1Var {
    public static void main(String[] args){
        if (args.length == 0)
            System.out.println("Nessun argomento");
        else
            for (int i=0; i<args.length; i++){
                System.out.println("argomento " + i + ": " + args[i]);
            }
    }
}
```

[fonte: 02x, sl. 6]

Anatomia — i tre costrutti nuovi.

1. `args.length` — «la proprietà pubblica `length` (read-only) contiene la dimensione **fisica** dell'array» [fonte: 02x, sl. 7]. *Read-only*: la leggi, non la assegni. *Fisica*: quante celle esistono, non quante hai usato. Senza parentesi — non è un metodo, e questo è il rovescio del §5: a un array non chiedi servizi.
2. `for (int i=0; ...)` — «come già in C99 (ma non in C89 e ANSI C), in Java e C# si possono definire variabili in ogni punto del programma. In particolare si può definire l'indice dentro al ciclo `for`, con **scope limitato al ciclo stesso**» [fonte: 02x, sl. 7]. Fuori dal ciclo `i` non esiste: è il primo esempio di scope che incontri nel corso.
3. `+` su stringhe — «NOVITÀ: l'operatore `+` concatena stringhe e nel farlo **converte anche in stringa ciò che stringa non è!**» [fonte: 02x, sl. 8]. È il motivo per cui `"argomento " + i` funziona con `i` intero, senza conversione esplicita. Tienilo a mente: è una conversione *implicita e sempre lecita*, l'opposto del §2 — verso stringa non si perde niente.

⚠ **Incoerenza nella fonte, da non copiare.** La slide 6 scrive «in Java e C#, da 0 a `args.length-1`»; le slide 7 e 8, con lo stesso codice, scrivono «da 0 a `args.length`». Il codice è l'arbitro: `i<args.length` significa che `i` arriva a `length-1`, e `args[args.length]` sarebbe fuori dall'array. Gli **indici validi** vanno da 0 a `length-1`; il **ciclo** si scrive con la condizione `i < args.length`.

Sono due modi di dire la stessa cosa, e la slide 6 dice quello giusto sugli indici.

Comandi:

```
# scrivi il file (usa il tuo editor, non un heredoc: qui il codice lo scrivi tu)
nano Esempio1Var.java
```

```
javac Esempio1Var.java
ls
java Esempio1Var alfa beta gamma
java Esempio1Var alfa "beta gamma"
java Esempio1Var
```

Anatomia dei comandi. `javac Esempio1Var.java` prende il **nome del file**, con estensione, e produce `Esempio1Var.class`. `java Esempio1Var` prende il **nome della classe che contiene il main**, senza estensione: la JVM cerca da sé il bytecode.

⚠ **Errore registrato il 16/09** (profilo/errori.md, FI2): a java avevi passato un file invece di una classe. Qui la prova è a portata di mano: dopo aver compilato, lancia anche `java Esempio1Var.class` e leggi il messaggio d'errore. Quello è il promemoria che ti serve.

Output atteso:

```
Argomento 0: alfa
Argomento 1: beta
Argomento 2: gamma
```

[fonte: 02x, sl. 12] — nota che la slide scrive Argomento con la maiuscola mentre il codice della slide 6 stampa argomento minuscolo: le due slide non sono allineate. Non inseguire la maiuscola.

Con `alfa "beta gamma"`: **due** argomenti, il secondo contenente uno spazio [fonte: 02x, sl. 12]. Le virgolette sono della shell, non di Java, e sono ciò che tiene insieme il secondo argomento.

Cosa verificare: senza argomenti stampa `Nessun argomento`; con tre argomenti ne stampa tre, numerati da 0; con `alfa "beta gamma"` ne stampa **due**. Se ne stampa tre, le virgolette non sono arrivate alla JVM.

Curiosità collegata — l'addendum 02z, da fare solo dopo il resto

Hai **JDK 21**, quindi l'addendum 02z lo puoi eseguire davvero. Java 21 introduce in *preview* forme più leggere di `main`: si possono omettere `public`, `static`, l'argomento `String[]` e perfino la classe che lo racchiude, «che sarà generata dal compilatore usando il nome del file» [fonte: 02z, sl. 4]. La versione minima:

```
// file ReMain.java
void main(){
    System.out.println("Hello world!");
}
```

[fonte: 02z, sl. 5]

```
javac --enable-preview --source 21 ReMain.java
java --enable-preview ReMain
javap ReMain.class
```

I due flag sono obbligatori: senza `--enable-preview` il compilatore rifiuta. `javap` è il **disassemblatore**: mostra la classe che il compilatore ha generato per te, e il docente lo usa proprio per farti vedere che esiste [fonte: 02z, sl. 5].

Perché lo fai per ultimo e perché non lo usi mai più: è una *preview feature*, e all'esame scrivi il main classico dentro una classe, in Eclipse. Serve a una cosa sola, ma importante: vedere che `public static void main(String[])` non è una formula magica: è una **convenzione di ricerca** del punto d'ingresso, e Java 21 ne accetta quattro forme (`public static void main(String[])`, `public static void main()`, `public void main(String[])`, `public void main()`) [fonte: 02z, sl. 7]. Omettere `static` «significa definire un metodo di istanza anziché una funzione statica», e allora il compilatore «crea un'istanza singleton della classe contenitrice e invoca su di essa il "nuovo main"» [fonte: 02z, sl. 4]. Questa frase risponde alla domanda di analisi 1 da un'altra direzione: `static` serve perché senza istanza non c'è oggetto su cui invocare — e se lo togli, qualcuno l'istanza la deve creare comunque.

§2 — Assegnamenti fra tipi numerici: il verso della perdita

Questo è il cuore della sessione. È il punto 1e del prossimo passo dichiarato in `stato/corrente.md`, ed è il bersaglio del pattern trasversale n. 5 promosso stamattina — *inversione del verso in una relazione asimmetrica*. Il 16/09 avevi detto: «mi è chiaro come funziona il meccanismo, ma non come riconoscere a livello di sintassi quando si perdono informazioni». Questo paragrafo è lì per quello.

Obiettivo: dato un assegnamento fra tipi numerici, dire **a occhio** se il compilatore lo accetta, e saper dire *da cosa* lo hai capito.

Concetto minimo. La regola del docente, alla lettera:

«In Java, C#, Scala sono ammessi **solo gli assegnamenti che non causano perdita di informazione**. Quindi, ad esempio: la frase `double x = 3.54F;` è **lecita** (da `float` a `double` non si perde precisione). La frase `float f = 3.54;` è **illecita** (da `double` a `float` si perderebbe precisione).» [fonte: 02x, sl. 21]

E i dati che servono per applicarla [fonte: 02x, sl. 14, 20]:

Tipo	Byte	Note
byte	1	−128 ... +127
short	2	−32768 ... +32767
int	4	$\approx -2 \cdot 10^9 \dots +2 \cdot 10^9$
long	8	$\approx -9 \cdot 10^{18} \dots +9 \cdot 10^{18}$ — le costanti terminano con L

Tipo	Byte	Note
float	4	≈ 6-7 cifre decimali significative — le costanti terminano con F
double	8	≈ 14-15 cifre decimali significative

Il procedimento, in due passi obbligatori e in quest'ordine. Non saltare il primo: saltarlo è precisamente l'errore del 16/09, dove il suffisso F — «il dato decisivo» — non è stato usato.

1. **Che tipo è il letterale a destra?** Lo dice la *sintassi*, non il valore:
 - senza punto decimale → `int` (42), a meno che finisca per L → `long` (42L);
 - con punto decimale → **double** (3.54), a meno che finisca per F → `float` (3.54F).
 - Questo è il punto: 3.54 è **un double**. Non è «un numero con la virgola», è un double.
2. **Poi confronta col tipo della variabile a sinistra:** la freccia va da destra a sinistra.
 - più **grande** → più **piccolo** = perdita ⇒ **rifiutato** (serve il cast);
 - più **piccolo** → più **grande** = nessuna perdita ⇒ **accettato**.

La contromisura contro l'inversione del verso (trasversale n. 5): non recitare la regola, *fai entrare il numero nella scatola*. `double x = 3.54F`; — un float da 4 byte che entra in una scatola da 8: ci sta, avanza posto. `float f = 3.54`; — un double da 8 byte in una scatola da 4: non ci sta, devi buttare via qualcosa, e Java non butta via niente senza che tu lo dica. Se non riesci a dire *quale delle due scatole è più grande*, non hai ancora il verso.

Il drill. Per ciascuna riga, scrivi su carta **A** (accettata) o **R** (rifiutata) e la *ragione in una riga*, nella forma «il letterale è X, la variabile è Y, X→Y ⇒ ...». **Poi** verifica in jshell, una riga per volta.

Le due righe 1-2 sono della fonte (sl. 21); le altre sono costruite sulla stessa regola.

```
double x = 3.54F;      // 1
float f = 3.54;        // 2
double y = 3.54;       // 3
float g = 3.54F;       // 4
long n = 42;           // 5
int i = 42L;           // 6
int k = 3.0;           // 7
double z = 42;         // 8
short s = 300;         // 9 ← prima di rispondere: 300 in uno short ci sta o no?
int m = 'A';          // 10
char c = 65;          // 11
byte b1 = 10;          // 12
byte b2 = 300;         // 13
short s2 = 40000;      // 14
```

jshell

Incolla **una riga alla volta** e leggi la risposta. Dove jshell accetta, stampa anche il valore: guardalo, a volte è più informativo dell'accettazione (righe 8, 10, 11).

Cosa verificare: non il numero di risposte giuste, ma che per ogni riga la tua ragione contenga **il tipo del letterale**. Se hai scritto «3.54 è un numero decimale», non hai fatto il passo 1.

Dove il procedimento si rompe. Tre righe del drill sono progettate per non obbedire alla lettura ingenua. Prima di guardare la risposta, riprovaci con il procedimento in mano:

Se sei bloccato — 1: una domanda, sulle righe 9-14

Le righe 9, 11, 12, 13 e 14 hanno **tutte la stessa forma**: un letterale `int` assegnato a una variabile di tipo più piccolo. Per il passo 2 dovrebbero avere tutte lo stesso esito. Ce l'hanno?

Se no, il passo 2 da solo non basta a spiegarle. Cosa sa il compilatore su 65, su 10 e su 300 che non sa su una variabile `int` qualunque? E: 300 dentro uno `short` ci sta; dentro un `byte`?

Se sei bloccato — 2: l'idea

Il compilatore tratta a parte le **espressioni costanti**: se il valore è noto a compile-time e ci sta nel tipo di destinazione, concede la conversione senza cast. Servono **entrambe** le condizioni, e questo spiega tutte e cinque le righe:

- `char c = 65;`, `byte b1 = 10;`, `short s = 300;` → costante **e** ci sta ⇒ accettati;
- `byte b2 = 300;`, `short s2 = 40000;` → costante ma **non** ci sta ⇒ rifiutati;
- `int q = 10;` `byte bb = q;` → ci starebbe, ma `q` è una **variabile** e il compilatore non ne segue il valore ⇒ rifiutato. Provala.

Nota che `short s = 300;` è **accettato**: il valore ci sta comodamente in 2 byte. Se hai risposto R perché «`int` è più grande di `short`», hai applicato il passo 2 senza guardare il valore — e qui è il valore a decidere.

Questo va **oltre la slide 21**: la regola del docente («solo gli assegnamenti che non causano perdita di informazione») è una formulazione sul *valore*, mentre il criterio che il compilatore applica di solito è sull'**ampiezza del tipo**. Le espressioni costanti sono l'eccezione in cui i due criteri coincidono davvero, e per questo sono ammesse. [oltre la fonte – verificalo in jshell prima di crederci]

Se sei bloccato — 3: il caso limite che smentisce la regola alla lettera

Prova anche questa, che non è nel drill:

```
float h = 123456789L;    // long (8 byte) → float (4 byte)
```

Un tipo da 8 byte verso uno da 4: per la regola della slide 21 dovrebbe essere rifiutato. Invece passa — e il valore stampato **non** è 123456789.

Perché: Java classifica `long` → `float` come conversione **allargante**, perché il *range* di `float` ($\approx 10^{38}$) copre quello di `long` ($\approx 10^{18}$). Ma `float` ha solo 6-7 cifre significative,

e quindi la precisione si perde comunque. Il criterio reale è il **range**, non il numero di byte né la perdita di informazione.

Non è una trappola d'esame: sui casi che il docente porta (float/double, int/long) la sua regola e il criterio reale danno lo stesso risultato. Serve a te, per una ragione sola: la tua contromisura del verso — *quale scatola è più grande* — qui va intesa come **quanto lontano arriva**, non *quante cifre tiene*. [oltre la fonte]

2b — Il cast, e come si chiama

Quando la conversione perde informazione e la vuoi comunque, la dichiari con un **cast**:

```
float f = (float) 3.54;
```

⚠ **Il termine del docente, che il 16/09 non hai usato** (profilo/errori.md, trasversale n. 4): il cast è la dichiarazione di **Design Intent**. Non è «dire al compilatore di stare zitto»: è mettere per iscritto che la perdita di precisione è una tua scelta di progetto, e quindi che la responsabilità del risultato è tua. Senza cast il compilatore ti protegge; con il cast gli stai dicendo che sai cosa stai facendo. Usa *questa* parola.

Verifica in jshell che (float) 3.54 è accettato e che (int) 3.9 dà 3 — non 4: il cast fra numerici **tronca**, non arrotonda. Per arrotondare servono Math rint e Math round (§5).

§3 — Overflow e aritmetica IEEE-754

Obiettivo: distinguere i casi in cui il compilatore ti salva da quelli in cui ti lascia sbagliare in silenzio, e sapere cosa fa Java quando dividi per zero.

Concetto minimo. Il docente mette in fila due esperimenti in C con il commento: «Il compilatore C è di bocca buona, accetta tutto. Però poi, a run time...» [fonte: 02x, sl. 23]. Il codice C della slide, da leggere non da compilare:

```
int main() {
    int a = 32767;
    int b = a+1;
    printf("%d, %d\n", a, b);
    short int c = 32767;
    short int d = c+1;
    printf("%d, %d\n", c, d);
}
```

[fonte: 02x, sl. 23]

E questo è il secondo:

```
int main() {
    char a = 125;
    printf("%d\n", a);
    a++; ++a;
```

```

    printf("%d\n", a);
    printf("%d\n", ++a);
}

```

[fonte: 02x, sl. 24]

Su Java il docente dà **solo la conclusione**, nelle due didascalie — le schermate degli esperimenti sono immagini e non stanno nel testo estratto, quindi **guardale sul PDF, slide 23-24**, e soprattutto rifai gli esperimenti tu:

- slide 23: «Il compilatore Java intercetta l'errore, **ma solo perché aveva convertito a+1 in int!**»
- slide 24: «E infatti stavolta, che non ci sono conversioni di mezzo, non si accorge del problema → **overflow**»

Il drill. Predici l'esito di ciascuna riga *prima* di eseguirla, poi verifica in jshell.

```

short c = 32767;
short d = (short)(c + 1);    // (a) con il cast
short e = c + 1;            // (b) senza cast
byte b = 125;
b++; ++b; b                 // (c) due incrementi: dove sei arrivato?
++b; b                      // (d) il terzo: e adesso?

```

I tre incrementi sono quelli della slide 24, dove il docente ne fa due e poi stampa ++a.

Cosa verificare, e perché è il punto del paragrafo. La riga (b) è **rifiutata**, mentre (c) e (d) sono **accettate** — e in (d) il valore che ottieni non è quello aritmetico: da 127 il passo successivo ti porta a -128. Il motivo sta nella didascalia del docente: `c + 1` è un'espressione di tipo `int` (le operazioni aritmetiche fra tipi piccoli promuovono a `int`), e `int` → `short` è un restringimento ⇒ serve il cast. L'operatore `++` invece **ha il restringimento incorporato**: non c'è nessuna conversione esplicita da rifiutare, e il compilatore lascia passare. Con il cast di (a), o con `++` di (c), il risultato **avvolge**.

△ **Pattern trasversale n. 1, caso limite.** «Java è tipizzato forte, quindi mi protegge dall'overflow» è falso, e (c) è il controesempio. Java ti protegge dalle **conversioni** implicite che perdono informazione, non dall'**overflow aritmetico**. Sono due garanzie diverse: la prima è a compile-time e c'è, la seconda sarebbe a run-time e non c'è.

3b — Divisioni per zero: reali contro interi

La fonte separa i due mondi, e la ragione è una sola [fonte: 02x, sl. 31]:

«I tipi interi **non seguono lo standard IEEE-754!** L'aritmetica degli interi dà errore in caso di divisioni per zero.»

Mentre per i reali «lo standard IEEE-754 incorpora le nozioni di **infinito** e **Not-A-Number (NaN)**. L'aritmetica dei reali perciò gestisce sia gli infiniti, sia le forme indeterminate!» [fonte: 02x, sl. 30].

Predici e verifica, in jshell:

```
5.0 / 0
-5.0 / 0
0.0 / 0.0
0.0 / -5
5 / 0
```

Cosa verificare: le prime quattro **producono un valore** e la sessione continua; l'ultima **interrompe** con un'eccezione. Leggi il nome dell'eccezione per intero: è il tuo primo incontro con un errore a run-time, e la distinzione failure/error del modulo 01 vive qui.

Sulla quarta riga il docente fa un'osservazione che val la pena registrare: «in matematica, 0/a fa sempre 0, senza segno — e in Java?» La risposta della slide: «**Java mantiene il segno nel risultato**, MA ciò non ne altera la semantica», mentre «C#, più correttamente, risponde semplicemente 0» [fonte: 02x, sl. 29]. Guarda cosa stampa jshell e chiediti perché «non ne altera la semantica» — cosa dà `-0.0 == 0.0`?

§4 — Caratteri, Unicode e UTF

Secondo bersaglio della sessione: il 16/09, alla domanda su U+1F608, hai risposto «4 char credo». Sono **2 char** e **4 byte** in UTF-8. Qui vedi da dove viene la differenza, con le mani.

Concetto minimo, dalla fonte [fonte: 02x, sl. 33]:

«In Java, i caratteri sono **Unicode (16 bit, UTF-16)**. Una costante carattere è racchiusa fra **apici singoli**. Un carattere può essere convertito: in `int` (o `short`, `long`) **con un cast**; in `String` con l'apposita funzione statica `String.valueOf`.»

E la frase che spiega tutto l'equivoco [fonte: 02x, sl. 34]:

«Si può ottenere facilmente la sequenza di byte UTF — bisogna però **partire da una stringa, NON da un carattere**. In Java si usa l'apposito metodo `getBytes(...)`.»

Perché quella frase è la risposta al tuo errore. `char` e `byte` non stanno sullo stesso piano. Un `char` è un'unità di codifica da 16 bit; i byte UTF sono il risultato di una **codifica** che si applica a una *stringa*. Non puoi chiedere i byte a un `char` — la fonte lo dice esplicitamente — e questo non è un dettaglio di API: è il segno che sono due livelli diversi. Unicode **numera** i caratteri (i *code point*); UTF-8/16/32 **mappano** i *code point* in sequenze di byte, con lunghezze diverse. Un *code point* fuori dal piano base non entra in 16 bit, e in UTF-16 occupa **due** `char` (una *coppia surrogata*).

Il drill, in jshell. Predici prima ogni risultato:

```
(int) 'A'
String.valueOf('A')
Character.toUpperCase('a')
"A".getBytes("UTF-8").length
"À".getBytes("UTF-8").length
```

```
"[U+1F608]".length()  
"[U+1F608]".getBytes("UTF-8").length  
"[U+1F608]".codePointCount(0, "[U+1F608]".length())
```

Cosa verificare: "[U+1F608]".length() ti dà il numero di char, getBytes("UTF-8").length il numero di byte, codePointCount il numero di caratteri Unicode veri. Tre numeri diversi per lo stesso carattere: sono i tre livelli. Se sai dire quale numero risponde a «quanti caratteri è» e quale a «quanto occupa», il punto d.5 di 02 è chiuso.

[le righe sull'emoji sono costruite sulla regola della sl. 33–34, per riprendere la verifica del 16/09; le prime tre sono della fonte]

4b — UTF-16 e il BOM: una tensione nella fonte

Il docente fa getBytes con tre codifiche e commenta: «UTF8 è chiaro. UTF32 anche. Ma... UTF16?» [fonte: 02x, sl. 34]. Provalo:

```
java.util.Arrays.toString("A".getBytes("UTF-8"))  
java.util.Arrays.toString("A".getBytes("UTF-16"))  
java.util.Arrays.toString("A".getBytes("UTF-32"))
```

Con UTF-16 escono **due byte in più davanti**. Sono il **BOM** (Byte Order Mark): «UTF-16 allows a Byte Order Mark, a code point with the value **U+FEFF**, to precede the first actual coded value» [fonte: 02x, sl. 35, citazione in inglese nella slide]. E: «i due byte alti indicano se la sequenza che segue è **big endian** (MSB first) o **little endian** (LSB first)».

△ **Due didascalie della fonte sono in tensione, e non le appiano io.**

La slide 34 annota «In hex: FE, FF è il marcatore “little endian”», mentre la slide 35 cita «**Everything in Java is stored in big-endian order**». Guarda i byte che ti ha stampato jshell e decidi tu quale delle due descrive ciò che vedi. Poi confronta con la slide 36, dove il docente fa lo stesso esperimento in C# e osserva che lì «si percepisce la struttura interna: chiedendolo si scopre che è little-endian (Intel)». Segnala l'esito in stato/giornata.md: se la slide 34 è imprecisa, è da sapere prima dell'orale, non da scoprire lì.

§5 — Primitivi contro classi-libreria: a chi si chiede il servizio

△ Questo paragrafo risponde direttamente all'errore del 16/09 su persone[].getMediaEta (profilo/errori.md, FI2). La domanda «a chi chiedo questo servizio?» è la stessa, e qui compare nella sua forma più semplice.

Concetto minimo [fonte: 02x, sl. 32]:

«In Java i tipi base sono **tipi primitivi, non tipi di oggetti** → non si può chiedere a un valore primitivo di “fare qualcosa”. Non si può chiedere al carattere 'A' quale sia la sua controparte minuscola, o se sia una vocale; analogamente, non si può chiedere all'intero 12 se sia pari. Invece che “fare domande” a un valore primitivo bisogna **chiamare una qualche funzione di una qualche libreria**.»

Le classi-libreria della fonte: `Character` offre `toUpperCase/toLowerCase`, `isWhiteSpace` e `digit` [fonte: 02x, sl. 37-38]; `Integer` offre `signum`, `rotateLeft/rotateRight` [fonte: 02x, sl. 39]; `Math` offre «funzioni statiche per qualunque calcolo», fra cui `rint` (arrotonda a intero, **risultato reale**), `round` (arrotonda a intero, **risultato intero**), `pow`, e le curiose `hypot` ($\sqrt{x^2+y^2}$) «senza errori di overflow/underflow intermedi») e `log1p` [fonte: 02x, sl. 26-28].

Il drill. Prima di eseguire, per ogni riga rispondi: *chi* fa il lavoro?

```
Character.toUpperCase('a')
Character.isWhitespace('\t')
Character.digit('8', 10)
Character.digit('8', 7)
Character.digit('B', 16)
Character.digit('B', 10)
Integer.signum(-42)
Integer.rotateLeft(6, 1)
Math.rint(2.5)
Math.round(2.5)
Math.sin(Math.PI/3)
Math.hypot(3, 4)
```

Anatomia di digit, che è la più istruttiva. «Il metodo `digit` restituisce il valore numerico di quel certo carattere nella base specificata; ovviamente il carattere dev’essere nel range fra 0 e `base-1`; altrimenti viene restituito convenzionalmente **-1** come segnale di errore» [fonte: 02x, sl. 38]. Gli esempi sono della slide: «il carattere '8' in base 10 o 16 indica il valore intero otto, mentre in base 7 **non esiste**; analogamente 'B' denota undici in base 16, ma non esiste in base 10.»

Il -1 è un *valore sentinella*: un errore comunicato come risultato ordinario, che il chiamante deve controllare. Tienilo a mente — è un modo di segnalare l’assenza che tornerà, e che il corso sostituirà più avanti con eccezioni e `Optional`.

Cosa verificare: `Math.rint(2.5)` e `Math.round(2.5)` non danno lo stesso tipo di risultato, e su 2.5 nemmeno lo stesso valore. Fermati su questo: perché due funzioni per «arrotondare»? Prova anche `Math.rint(3.5)`.

Se sei bloccato sul perché `rint(2.5) ≠ round(2.5)`

Sono due politiche di arrotondamento diverse, e i tipi di ritorno lo suggeriscono. La differenza si vede solo sui valori esattamente a metà, e c’è una politica che evita di introdurre un bias sistematico quando arrotondi molti numeri. Prova `rint` su 0.5, 1.5, 2.5, 3.5 e guarda la sequenza dei risultati: la regola salta all’occhio. [oltre la fonte: la slide 27 dà solo le due definizioni]

5b — Il confronto con gli altri linguaggi, per l’orale

Non è codice da scrivere, è una distinzione da possedere [fonte: 02x, sl. 32, 40-42]:

- **Java:** tipi primitivi + classi-libreria di nome simile (`Integer`, `Character`). «NON VANNO CONFUSE COI TIPI-OGGETTO DI Scala e Kotlin!»

- **C#:** «la differenza è più sfumata — i tipi primitivi in realtà sono classi, ad esempio `int` è in realtà un sinonimo per la classe `Int32`». Quindi puoi scrivere `ch.ToString()` (metodo) invece di `Char.ToString(ch)` (funzione statica), e perfino `'B'.ToString()`.
- **Scala e Kotlin:** «la dicotomia si supera: i tipi primitivi non esistono più, al loro posto ci sono veri tipi-oggetto». Le costanti numeriche «sono veri oggetti». In Scala `toFloat` non vuole le parentesi («principio di accesso uniforme»), in Kotlin sì [fonte: 02x, sl. 43].

△ **Tre livelli vicini da non collassare** (trasversale n. 1): tipo primitivo (Java) → primitivo che è una classe (C#) → tipo-oggetto pieno (Scala/Kotlin). Il caso limite che li separa è una riga sola: `'B'.ToString()` si scrive in C#, non in Java. E la classe `Character` di Java non è un tipo-oggetto: è una **libreria di funzioni statiche** che sta *accanto* al primitivo, non al suo posto.

5c — Da stringa a numero, e cosa succede se la stringa è sbagliata

Serve per il §6, dove gli argomenti arrivano come `String` [fonte: 02x, sl. 44–45]:

- **Java:** `Integer.parseInt`, `Double.parseDouble`.
- Scala/Kotlin: `toInt`, `toDouble`; e nelle versioni «sfortunato-aware», `toDoubleOption` (Scala) e `toDoubleOrNull` (Kotlin).
- «MA questi metodi pretendono stringhe **giuste**! Altrimenti... **BOOM!**» E in Java «per evitare tale errore si può **solo controllare la stringa prima di convertirla**».

Provalo, e leggi il nome dell'eccezione:

```
Double.parseDouble("3.54")
Double.parseDouble("aa")
```

Quel «solo controllare prima» è il vincolo con cui affronti l'ultimo caso di collaudo del §6.

§6 — L'esercizio completo: equazioni di 2° grado

Obiettivo: un programma da riga di comando che risolve $ax^2 + bx + c = 0$, collaudato sui casi del docente. È il primo esercizio che chiude un modulo: vale come **unità di verifica** solo se lo scrivi **a freddo, senza soluzione sotto mano** (CLAUDE.md §7.2).

La specifica, dalla fonte [fonte: 02x, sl. 46]:

«Fornire i coefficienti `a`, `b`, `c` dalla riga di comando — diventano i tre argomenti (stringa) `args[0]`, `args[1]`, `args[2]`, **ma sono stringhe, non numeri!** L'algoritmo è ben noto: per estrarre la radice, `Math.sqrt`; ovviamente, vanno distinti i vari casi: radici reali, immaginarie, etc.»

Il problema di stampa che il docente segnala [fonte: 02x, sl. 46]: per le soluzioni immaginarie «in matematica di solito si scrive $3 \pm 2i$ — servirebbe il carattere $\pm$: esiste? Sì, MA la sua codifica dipende dal set di caratteri in uso. Nel prompt comandi: codice

00B1 (in decimale, 177) '±'. Nelle finestre Windows: codice 00F1 (in decimale, 241) 'ñ'.»

Sul tuo terminale Linux in UTF-8 usa '±'. La nota su 00F1 riguarda la code page di Windows e non ti tocca — ma è la prova pratica che «quale byte è quel carattere» dipende dalla codifica, cioè il §4 applicato a un caso reale. Verificalo: `System.out.println('±')`.

6.1 Cosa pensare prima di scrivere — domande, non risposte

Il docente non dà l'analisi: la devi fare tu. Rispondi per iscritto, poi scrivi il codice.

1. Quanti argomenti ricevi, e cosa fai se sono meno di tre? (§1: `args.length`)
2. Come trasformi `args[0]` in un numero, e di quale tipo — `int` o `double`? Da cosa lo decidi?
3. Il discriminante: quali casi distingue, e quanti rami ti servono? Il docente ne nomina tre («soluzioni reali distinte, soluzioni immaginarie, soluzioni coincidenti»), e nel collaudo ne compaiono altri due.
4. Con $a = 0$ l'equazione non è di secondo grado. Cosa fa la tua formula se dividi per $2a$? Il §3b ti ha già mostrato *esattamente* cosa: quale dei due comportamenti otterrai, quello dell'aritmetica reale o quello dell'intera?
5. Con $a = 0$ e $b = 0$: c'è un'equazione da risolvere?
6. Cosa stampi, esattamente? «Partiamo dall'**output atteso**» [fonte: 02x, sl. 47]: scrivi su carta le righe che vuoi vedere, **prima** del codice.

△ **Pattern trasversale n. 2 — fermarsi al primo indizio.** Il primo java Equazioni 1 5 6 che stampa due radici giuste non chiude l'esercizio. Sotto ci sono otto casi di collaudo, e almeno tre non passano dalla formula standard. L'esercizio è chiuso quando **ogni** caso della lista 6.2 ha un esito che hai previsto.

6.2 Il contratto: il collaudo del docente

Non c'è startkit, quindi non c'è una classe di test: **questa lista è il contratto**. Il docente la introduce così [fonte: 02x, sl. 48]:

«Il collaudo: una serie di affermazioni “mi aspetto che...”, studiate per coprire tutti i casi (soluzioni reali distinte, soluzioni immaginarie, soluzioni coincidenti — **occhio agli errori numerici...**)»

I casi, alla lettera dalla slide 48. **Riempi tu la colonna di destra, a mano e prima di eseguire** — la slide li lascia aperti di proposito, e le due voci annotate sono le uniche che il docente commenta:

#	Argomenti	Cosa ti aspetti
1	1 5 6	
2	1 -5 6	
3	1 0 1	
4	1 0 -1	
5	1 0 0	

#	Argomenti	Cosa ti aspetti
6	0 0 1	«Impossibile» [fonte: sl. 48]
7	0 1 -2	« $x = 2$ (1° grado)» [fonte: sl. 48]
8	1 0 aa	«.. e se gli argomenti NON sono numerici?»

Come eseguire il collaudo:

```
javac Equazioni.java
java Equazioni 1 5 6
java Equazioni 1 -5 6
java Equazioni 1 0 1
java Equazioni 1 0 -1
java Equazioni 1 0 0
java Equazioni 0 0 1
java Equazioni 0 1 -2
java Equazioni 1 0 aa
```

Cosa verificare: per ciascuno, l'esito coincide con quello che avevi scritto in tabella. Dove non coincide, **prima** capisci perché, poi correggi. Se il caso 8 termina con un'eccezione, hai appena incontrato dal vivo il «BOOM!» della slide 44, e il rimedio è quello della slide 45: controllare la stringa prima di convertirla.

Il caso 3 (1 0 1, radici immaginarie) e il caso 5 (1 0 0, radici coincidenti) sono quelli dove il docente mette l'avvertimento sugli **errori numerici**: un discriminante che in matematica è esattamente 0 può risultare un numerino minuscolo diverso da zero in double. Chiediti se `disc == 0` è un confronto affidabile — e collega la risposta alle «14-15 cifre significative» della slide 20.

Se sei bloccato — 1: una domanda

Il tuo programma ha bisogno di distinguere dei casi. Quanti **rami indipendenti** ti servono, e in che **ordine** devi controllarli? C'è un controllo che, se lo metti per secondo, rende inutile il primo.

Guarda i casi 6 e 7 della tabella: hanno lo stesso a. Cosa devi aver già deciso, prima di guardare il discriminante?

Se sei bloccato — 2: l'idea

Il docente stesso mette in fila la scaletta, nelle didascalie del collaudo: il grado dell'equazione si decide **prima** del discriminante, perché con $a = 0$ la formula del discriminante non descrive più il problema. Dentro il caso $a = 0$, il valore di b separa l'equazione di primo grado dal caso impossibile. Solo nel ramo $a \neq 0$ calcoli il discriminante, e là i casi sono tre, per segno.

Per la stampa delle immaginarie, la parte reale e il modulo della parte immaginaria si ricavano dalla stessa formula che usi per le reali, cambiando cosa metti sotto radice. Il carattere lo hai: '±'.

Nessun codice: la traduzione in Java è il tuo lavoro. [l'ordine dei controlli è dedotto dalla lista di collaudo delle sl. 48, non dalla soluzione]

△ **Le slide 49-51 contengono la soluzione del docente.** Non aprirle. La slide 49 dice «Sicuro/a di voler vedere la soluzione...? Ti farebbe MOLTO meglio svilupparla e collaudarla da solo/a...», e ha ragione: questo è il primo esercizio che può chiudere un modulo, e si chiude solo a freddo. Apri le 49-51 **dopo** che tutti gli otto casi passano — istruzioni al §7.

§7 — Dopo il collaudo verde: confronto con la soluzione

Solo ora apri le slide 49-51. La slide 50 è marcata dal docente come «soluzione **imperfetta**» e la 51 come «soluzione corretta (con finezze di stampa)»: leggile **in quest'ordine** e cerca la differenza fra le due, perché quella differenza è la lezione.

Cosa guardare nel confronto — non cosa c'è, ma dove confrontare:

- **L'ordine dei controlli:** il docente decide il grado prima o dopo il discriminante? Coincide con quello che hai scelto tu?
- **Il caso degenerare:** come tratta $a = 0$, e come lo distingue da $a = 0, b = 0$?
- **Gli argomenti non numerici:** controlla prima o intercetta dopo?
- **La stampa:** quante cifre mostra, e come formatta le immaginarie. Le «finezze di stampa» della 51 sono la parte che si vede all'esame.
- **La divisione in metodi:** tutto nel main, o qualcosa fuori?

Ciò che il tuo codice non aveva va in stato/giornata.md; se è un pattern, in profilo/errori.md.

Leggere un errore

Non ci sono test JUnit qui, quindi la triade del modulo 01 si presenta in una forma più cruda. Da riprendere a ogni esecuzione:

- **javac dà error:** → nessun .class prodotto, non esegui niente. È il §2: quasi sempre un assegnamento che perde informazione, e il messaggio lo dice (incompatible types: possible lossy conversion from double to float). Leggi *i due tipi* nel messaggio: sono il passo 1 e il passo 2 del procedimento, scritti dal compilatore.
- **compila e l'output è sbagliato** → è l'analogo della **failure**: l'errore è nella logica, e il caso di collaudo che l'ha scoperto ti dice dove cercare.
- **compila ed esplode a run-time** → è l'analogo dell'**error**: ArithmeticException (§3b), NumberFormatException (§5c), ArrayIndexOutOfBoundsException (§1, se leggi args[2] senza controllare args.length). Leggi lo **stack trace** dall'alto: la prima riga dice cosa, la seconda dove.

△ profilo/errori.md, FI2: «compilazione ed esecuzione fuse», error collocato «nella struttura» invece che nell'esecuzione. Qui le tre categorie hanno tre comandi diversi che le producono: se l'errore è uscito da javac è del primo tipo, se è uscito da java è degli altri due. Guarda **quale comando** ha parlato.

Come chiudere il modulo

02x è l'unità di verifica di 02, di tipo **esercizi** (CLAUDE.md §7.2). Il modulo è chiuso quando:

- ☐ Equazioni.java compila e tutti gli **otto** casi del §6.2 danno l'esito che avevi previsto in tabella **prima** di eseguirli;
- ☐ l'hai scritto **senza la soluzione sotto mano** (slide 49-51 chiuse fino al §7);
- ☐ sul drill del §2 sai dire, per ogni riga, **il tipo del letterale** e non solo l'esito;
- ☐ sai rifare a voce i tre punti deboli di 02: conversioni fra reali con il verso giusto (§2), char contro byte (§4), e javac/java con lo strumento giusto per ciascuno (§1).

Il quarto punto è la ridomanda a voce annunciata nel prossimo passo di stato/corrente.md: chiedimela quando i primi tre sono fatti.

Connessioni

- **Con il modulo 02**, che questa esercitazione chiude: le slide 2-5 sono il riassunto dichiarato di 02, e le slide 20-21 sono i tipi base su cui la verifica del 16/09 si era fermata (d.4, d.5). Il punto che 02 aveva lasciato aperto e che 02x chiude è **perché il main è static**: «deve esistere dall'inizio alla fine del programma» [fonte: 02x, sl. 2].
- **Con il modulo 01** (chiuso): «in Java non si include nulla (**link dinamico**): si invoca il servizio `println` del componente `System.out`» [fonte: 02x, sl. 5] è la sl. 83 di 01 vista da dentro il codice. E `-1` di `Character.digit` come segnale d'errore è il primo esempio del «collaudo dei casi critici» della sl. 13.
- **Con LAB01** (prossimo gradino pratico, prerequisiti 02 e 03): riusa `javac/java` da riga di comando di questo §1 e aggiunge `javadoc`, `jar cmf` e i test con `assert` e `-ea`. La libreria `MyMath` di LAB01 è lo stesso schema di `Math` del §5 — funzioni statiche in una classe libreria — ma scritta da te.
- **Con LAB04** (FrazLib): la distinzione «operazione di un solo soggetto → metodo del soggetto; operazione fra più soggetti alla pari → funzione statica di libreria» è quella del §5, e il tuo errore del 16/09 su `persone[].getMediaEta` è lo stesso di chiedere a `'A'` la sua maiuscola.
- **Con il modulo 16** (*Wrapper per tipi primitivi*): `Integer` e `Character` del §5 sono le classi-libreria; in 16 diventano wrapper, con autoboxing. Quando ci arrivi, la domanda da farsi è se `Integer` in 16 è ancora «solo una libreria di funzioni statiche» come lo è qui.
- **Con il modulo 36** e ES-NAN: `NaN` e gli infiniti del §3b tornano lì come **risultato assente** (`Double.NaN` per un determinante che non esiste). Qui li vedi nascere.
- **Con RETI (S2)**: il BOM e l'endianness del §4b sono lo stesso problema del *network byte order*. «Quale byte viene prima» è una convenzione da concordare, e chi non la concorda legge numeri sbagliati.